

FSCK-NN

Reference-temperature theory, database format, and PyTorch training

Project implementation guide

Based on the current FSCK-NN source code

This guide describes the existing 32-point FSK V4 database and the **kappa-only** training pipeline. The network has seven physical inputs and 32 kappa outputs. No a-function targets or intermediate CSV files are used. The final section explains why reference temperature remains an input.

1 FSCK Database file names and format

1.1 Directory and filename convention

The default database directory is

```
/Volumes/home/cnp10/CODES/radiationDatabase/FSKTableV4
```

This default is defined by `DEFAULT_DB` in `training/fsck_backend.py`:

```
DEFAULT_DB = Path('/Volumes/home/cnp10/CODES/radiationDatabase/FSKTableV4')
```

The training script `training/train_fsck.py` imports this constant and uses it as the default for its command-line argument:

```
p.add_argument('--database', type=Path, default=DEFAULT_DB)
```

To use another database without editing the Python files, pass `-database`. From the `FSCK_NN` directory, run:

```
python3 training/train_fsck.py --database /path/to/FSKTableV4
```

Each pressure has a subdirectory, such as `01.0` for 1 atm. One file identifies a fixed pressure and composition. For example:

```
01.0/P.01.0_C02.0.00_H20.0.00_C0.0.00_fv.1.E-05_k.dat
```

The general pattern is

```
P.<P>_C02.<xCO2>_H20.<xH20>_C0.<xCO>_fv.<fv>_k.dat
```

Token	Meaning and database range
P	Pressure in atmospheres, 0.1–80 atm. 34 grid values: 0.1, 0.2, 0.3, 0.4, 0.5, 0.7, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80.
CO2	Carbon dioxide mole fraction, 0–1. 10 grid values: 0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.25, 0.5, 0.75, 1.
H2O	Water vapor mole fraction, 0–1. 13 grid values: 0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.1, 0.15, 0.2, 0.25, 0.5, 0.75, 1.
CO	Carbon monoxide mole fraction, 0–0.5. 6 grid values: 0, 0.01, 0.05, 0.1, 0.25, 0.5.
fv	Dimensionless soot volume fraction, 0– 10^{-5} (0–10 ppm by volume). 4 grid values: 0, 10^{-7} , 10^{-6} , 10^{-5} .
_k.dat	Binary kappa table

The prefix separator and decimal point are distinct: **P.01.0** contains the prefix **P.** followed by the number **01.0**. The Python reader parses these values from the filename and checks that the pressure agrees with its directory.

The pressure and composition ranges and grid values in the table come from **PDB**, **xC02DB**, **xH20DB**, **xC0DB**, and **fvDB** in **fskTableV4.f90**. They describe the full declared database axes; **loadIntoMem** may load a subset according to the requested limits and available files. The listed gas mole fractions must sum to at most unity for a physical mixture.

1.2 Temperature dimensions and record ordering

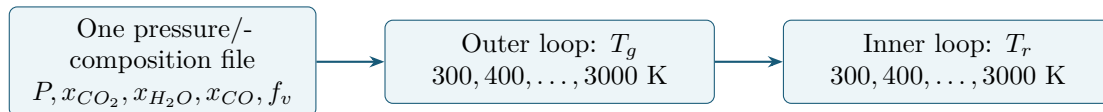
Each file contains every pair of local gas temperature T_g and reference Planck-weighting temperature T_r on the grid

$$T_i = 300 + 100(i - 1) \text{ K}, \quad i = 1, \dots, 28.$$

Therefore there are $28 \times 28 = 784$ binary records per file. The reference temperature varies fastest. With one-based indices,

$$r = 28(i_g - 1) + i_r, \quad 1 \leq r \leq 784.$$

The reference temperature remains an NN input because the cumulative Planck-weighted k-distribution depends on it, even when no a-function is trained.



Records 1–28: $T_g = 300$ K; records 29–56: $T_g = 400$ K;
records 57–784: $T_g = 3000$ K.

Figure 1: Temperature-pair ordering within each database file.

1.3 Binary record layout

The existing reader uses Fortran unformatted *direct-access* records. There is no text header and no sequential-unformatted record marker. Each record contains 33 single-precision floating-point values:

$$[\kappa_1, \dots, \kappa_{32}, \kappa_{\max}].$$

The first 32 values correspond to fixed quadrature nodes. The last is the maximum-kappa value, not an additional quadrature point. The expected file size is

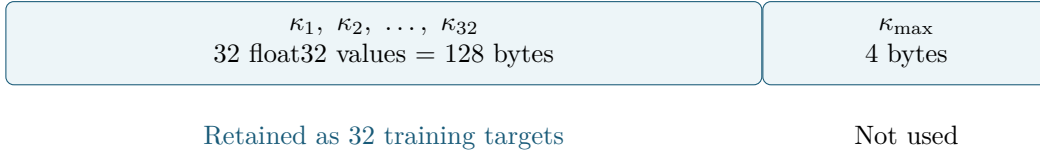


Figure 2: One 132-byte record; byte offset of record r is $132(r - 1)$.

$$784 \times 33 \times 4 = \boxed{103,488 \text{ bytes}}.$$

In short each set of $(P, x_{CO_2}, x_{H_2O}, x_{CO}, f_v, T_g, T_r)$ will give 32 κ values to train with (excluding the $\kappa_{\max}$).

2 Train using the FSCK database

2.1 Project layout and requirements

```

FSCK_NN/
  fortran/                # preserved numerical sources and row_bridge.inc
  training/
    train_fsck.py         # PyTorch model, batches, training, checkpoints
    fsck_backend.py       # binary reader and Fortran build/ctypes bridge
    pressure_selection.py  # pressure folder selection
    requirements.txt       # PyTorch and ONNX dependencies
    build/                # generated Fortran module and shared library
    runs/                 # training output directories
  test/
    compare_fsck.f90      # NN / direct-database comparison
    nn_model.f90          # Fortran ONNX interface
    onnx_bridge.c         # ONNX Runtime C API bridge
    comparison_grid.f90   # read the Fortran grid namelist
    grid.nml              # pressure/composition/temperature grid
    build.sh              # compile with external ONNX Runtime SDK
    plot_results.py       # plot the completed comparison CSV
    requirements.txt       # NumPy and Matplotlib for plotting
  docs/
    fsck_nn_guide.tex     # this document

```

Requirements are Python 3.9 or later, gfortran, PyTorch, and ONNX. The current build flags target gfortran; Intel Fortran is not configured by these scripts. If the Python packages are not installed already, then from the FSCK_NN directory, install the Python dependencies:

```
python3 -m pip install -r training/requirements.txt
```

A working Fortran compiler must be on `PATH`, or supplied using `-compiler`. OpenFOAM is not needed to run this training program.

2.2 Direct database-to-PyTorch workflow

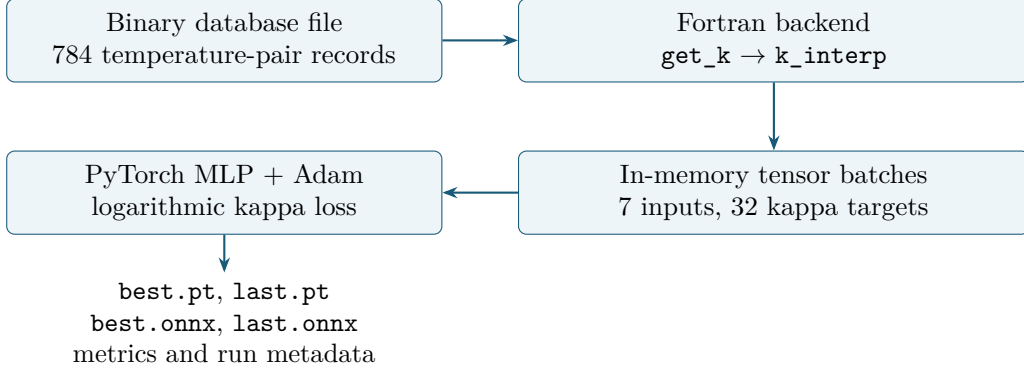


Figure 3: Training uses database records directly; no CSV dataset is created.

The build injects `row_bridge.inc` into a generated copy of the V4 module. A shared library is loaded through Python `ctypes`. For each file, the bridge loads all temperature pairs into singleton pressure and composition axes, then follows the kappa-only path of `fsk_v4_table`. It does not calculate `afun`. Computed kappa values are checked against the first 32 stored values at each grid record.

2.3 Inputs, network, and loss

The physical input vector is

$$\mathbf{x} = (P_{\text{atm}}, x_{CO_2}, x_{H_2O}, x_{CO}, f_v, T_g, T_r),$$

and the target vector is $(\kappa_1, \dots, \kappa_{32})$. The default MLP has three hidden layers of 128 units with SiLU activations. Input preprocessing is part of the model:

$$\begin{aligned} P' &= \frac{\log_{10} P + 1}{\log_{10} 800}, & f'_v &= \frac{\ln(1 + f_v/10^{-9})}{\ln(1 + 10^{-5}/10^{-9})}, \\ T'_g &= \frac{T_g - 300}{2700}, & T'_r &= \frac{T_r - 300}{2700}. \end{aligned}$$

Mole fractions are unchanged. These scales are fixed and do not use validation statistics. Input values are not clipped to these nominal ranges.

The network predicts $z_i = \log_{10} \kappa_i$. Training minimizes

$$\mathcal{L} = \frac{1}{32B} \sum_{b=1}^B \sum_{i=1}^{32} [z_{b,i} - \log_{10}(\max(\kappa_{b,i}, \kappa_{\text{floor}}))]^2, \quad \kappa_{\text{floor}} = 10^{-8}.$$

The floor is in native database units and handles zero kappas. Values below it are not distinguished by the loss. `forward()` returns logarithmic predictions, while `predict_kappa()` returns 10^{z_i} . No constraint forces predictions to be monotonic across quadrature nodes.

2.4 Training commands

All commands below run from the `FSCK_NN` directory. Start with a small pipeline check; the first six sorted files are not a representative training dataset.

```
python3 training/train_fsck.py \  
  --pressure 1 --limit-files 6 --epochs 2 \  
  --output training/runs/my_smoke
```

Train using all files at 1 atm:

```
python3 training/train_fsck.py \  
  --pressure 1 --epochs 20 \  
  --output training/runs/one_atm
```

Train using every pressure and composition:

```
python3 training/train_fsck.py \  
  --pressures all --epochs 20 \  
  --output training/runs/full_database
```

Select existing pressure folders with `-pressures`. Values are in atm; `01.0` and `1` select the same pressure. Use `all` (the default), a single value, an inclusive range `low:high`, or a comma-separated list. Values and ranges may be combined:

```
python3 training/train_fsck.py --pressures 1:5  
python3 training/train_fsck.py --pressures 0.5,1,3  
python3 training/train_fsck.py --pressures 0.5,2:5
```

Ranges select existing folders, not a generated pressure grid. For example, `1.5:3` includes 2 and 3 atm folders if present. Duplicate selections are included once. Each requested value or range must match at least one folder. Invalid numbers, reversed ranges, and unmatched selections produce errors. The original `-pressure 1` option remains an alias. Actual folders used are printed and recorded in `run.json`. The `-limit-files` cap is applied after filtering across all selected pressures, so a small cap can include only the lowest selected pressure.

Use a new or empty output directory for each run. To use another database, add `-database` followed by the database path. Without `-output`, a timestamped folder is created under `training/runs`.

Option	Default	Purpose
<code>-database</code>	DEFAULT_DB	Database root directory; defaults to <code>/Volumes/home/cnp10/CODES/radiationDatabase/FSKTableV4</code> , defined in <code>fsck_backend.py</code>
<code>-limit-files</code>	No limit	First N sorted files after pressure filtering, shared across training and validation. For example, 6 files give 4,704 samples; use for quick pipeline checks
<code>-pressures</code>	all	Pressure value, range, or list
<code>-epochs</code>	20	Complete training/validation passes
<code>-batch-size</code>	1024	Rows per optimizer batch
<code>-chunk-files</code>	8	Files loaded together for row shuffling
<code>-width, -depth</code>	128, 3	Hidden-layer width and count
<code>-learning-rate</code>	10^{-3}	Adam learning rate
<code>-kappa-floor</code>	10^{-8}	Lower floor for log targets
<code>-validation-fraction</code>	0.1	Fraction of files held out
<code>-seed</code>	42	Reproducible split and shuffling
<code>-threads</code>	4	PyTorch CPU thread count
<code>-device</code>	auto	CUDA when available, otherwise CPU
<code>-compiler</code>	gfortran	Fortran compiler command
<code>-debug</code>	off	Fortran bounds/runtime checks

Explicit device choices are `cpu`, `cuda`, and `mps`. Use `-help` to list all options. At least two selected files are required.

2.5 Batching and validation

The program splits entire files, not individual records. Every temperature pair from a file belongs to the same split. The validation file count is the rounded requested fraction, bounded to at least one file and at most $N - 1$. Each epoch shuffles training files and rows within bounded file groups. Fortran calls are sequential because the module contains mutable state; it is not called concurrently from multiple data-loader workers.

All selected rows are revisited every epoch; the program does not cache the full database in RAM. Validation evaluates held-out files without gradient updates. The split can include similar compositions, or the same composition at another pressure, on both sides. It measures held-out-file accuracy, not strict extrapolation to unseen composition families.

2.6 Checkpoints, metrics, and inference

Output	Contents
<code>best.pt</code>	Checkpoint with lowest validation log-space MSE
<code>last.pt</code>	Checkpoint from the most recently completed epoch
<code>best.onnx</code>	Kappa inference model matching <code>best.pt</code>
<code>last.onnx</code>	Kappa inference model matching <code>last.pt</code>
<code>metrics.jsonl</code>	Per-epoch log-space MSE, MAE, and row counts
<code>run.json</code>	Input/output names, file splits, settings, and Fortran hash

Checkpoints store model parameters, preprocessing buffers, model configuration, optimizer state, epoch, and fixed quadrature metadata. Automatic resumption of training is not implemented. The fixed quadrature weights saved in metadata are not learned targets.

ONNX models are exported automatically whenever the corresponding checkpoint is saved; no additional command-line option is required. Both files are written in the directory selected by `-output`. For example, the `smoke-test` command produces `training/runs/my_smoke/best.onnx` and `training/runs/my_smoke/last.onnx`. They use opset 17 and accept a float32 tensor named `physical_inputs` with shape $(B, 7)$, in the same physical input order as the PyTorch model. The batch dimension B is dynamic. Preprocessing and the conversion 10^{z_i} are included, so the output `kappa`, with shape $(B, 32)$, contains kappa in native database units rather than logarithms. The `onnx` package is included in `training/requirements.txt`. The exports contain inference models; optimizer state and quadrature metadata remain in the PyTorch checkpoints.

3 Test the NN model using Fortran

The `test/` directory compares NN kappa and stretching function a against direct V4 database access. Model inference, grid generation, database interpolation, and both a-function calculations run in the compiled Fortran program. Python is used only to plot the completed CSV.

3.1 Build and run

Compilation requires gfortran, a C compiler, and the ONNX Runtime C/C++ SDK, which is not bundled. Install the SDK matching your platform and set `ONNXRUNTIME_ROOT` to its extracted directory, which must contain `include/` and `lib/`. From `FSCK_NN`, run:

```
bash test/build.sh
mkdir -p test/results/new_onnx_comparison
test/build/compare_fsck training/runs/my_smoke/best.onnx \
  test/grid.nml \
  /Volumes/home/cnp10/CODES/radiationDatabase/FSKTableV4 \
  test/results/new_onnx_comparison/comparison.csv
```

The four arguments are the ONNX model, grid namelist, database root, and output CSV. Existing output files at the selected paths are overwritten. Trained checkpoints are not bundled. Run the smoke-training example first, or supply your own ONNX model. Training exports ONNX files automatically.

The build defaults to `test/vendor/onnxruntime-osx-arm64-1.20.1`, resolved relative to `build.sh`, independently of the working directory. If the SDK is installed there, no environment variable is required. An explicitly set `ONNXRUNTIME_ROOT` overrides this default.

3.2 Fortran calls to ONNX Runtime

The module `test/nn_model.f90` exposes `read_model`, `predict`, and `close_model`. It calls the C-compatible bridge in `test/onnx_bridge.c` through `ISO_C_BINDING`. ONNX Runtime loads the graph once and executes it for each prediction. No neural-network weights or activations are reimplemented in Fortran.

```
use iso_fortran_env, only: real64
use nn_model, only: read_model, predict, close_model
```

```

real(real64) :: state(7), kappa(32)

state = [1d0, .1d0, .2d0, .05d0, 1d-6, 1000d0, 1500d0]
call read_model('training/runs/my_smoke/best.onnx')
call predict(state, kappa)
call close_model()

```

The wrapper converts the seven physical inputs to float32 and receives 32 float32 kappa values, which it returns as Fortran double precision. Preprocessing and conversion from logarithms are already in the ONNX graph. The graph must use the training export's `physical_inputs` and `kappa` tensor names and the original 32-point quadrature convention.

3.3 Comparison grid and stretching function

Edit `test/grid.nml` to specify the Cartesian-product grid:

```

&grid
  pressures = 1.0, 1.5
  co2 = 0.015, 0.035
  h2o = 0.015, 0.035
  co = 0.005, 0.025
  soot = 5e-8, 5e-7
  tgas = 950, 1450
  tref = 1100, 1600
/

```

This example gives 128 states, including values between database nodes. Pressure is in atm, gases are mole fractions, soot is volume fraction, and temperatures are in K. Values outside the database ranges are rejected. The original loader may require many bounding files, so begin with a small grid.

Direct access calls `loadIntoMem` and `get_ka` in the preserved `fskTableV4.f90`. The NN is evaluated through ONNX at both (T_g, T_r) and (T_g, T_g) . Both curves are passed to the same `afun` routine used by direct access. The curves are not sorted or renormalized. Nonmonotone NN curves violate `afun`'s assumptions, so their a-values are flagged as diagnostic only.

3.4 1:1 plots and verification

After the Fortran executable finishes, use Python only for plotting:

```

python3 -m pip install -r test/requirements.txt
python3 test/plot_results.py \
  test/results/new_onnx_comparison/comparison.csv

```

Outputs include `k_parity.png` and `a_parity.png` (also PDFs) and `summary.json`. Each scatter point compares the same state and quadrature node, with a dashed 1:1 line and linear and logarithmic views. Negative values remain in linear plots; only positive pairs appear on log axes. Nonmonotone NN curves are shown in orange as diagnostic only. Comparison results are generated locally and are not bundled.

See `test/README.md` for SDK setup and full interface details.

4 A little background on FSCK

4.1 Local temperature and Planck-weighting temperature

Full-spectrum correlated- k (FSCK) methods replace integration over spectral position by integration over a cumulative absorption-coefficient coordinate. Let η denote wavenumber and let

$$\phi = (P, x_{CO_2}, x_{H_2O}, x_{CO}, f_v, T_g)$$

be the local thermodynamic state. The spectral absorption coefficient $\kappa_\eta(\phi)$ depends on this state, including the local gas temperature T_g . Write the blackbody spectral intensity as $I_{b\eta}(T)$, with integrated intensity

$$I_b(T) = \int_0^\infty I_{b\eta}(T) d\eta = \frac{\sigma T^4}{\pi}.$$

The reference temperature T_r specifies the Planck spectrum used to weight and accumulate the absorption coefficients. It does not replace T_g in κ_η , and it is typically calculated as a domain average temperature.

In a nonhomogeneous medium, radiation emitted in one region travels through regions with different temperatures and compositions. If each region uses its own T_g for Planck weighting, the cumulative spectral weights change along the path, so the same numerical g does not generally represent the same spectral contribution in neighboring regions. A common T_r supplies a fixed Planck-weighting convention for relating these local distributions within the correlated- k formulation. The local T_g still determines the absorption spectrum and thermal emission; the a -function accounts for the difference between local and reference Planck weights.

4.2 Planck-weighted distribution and cumulative coordinate

For a fixed local state, define the distribution

$$f(k; \phi, T_r) = \frac{1}{I_b(T_r)} \int_0^\infty I_{b\eta}(T_r) \delta(k - \kappa_\eta(\phi)) d\eta, \quad (1)$$

where δ is the Dirac delta. Its cumulative distribution is

$$g(k; \phi, T_r) = \frac{1}{I_b(T_r)} \int_{\{\eta: \kappa_\eta(\phi) \leq k\}} I_{b\eta}(T_r) d\eta, \quad 0 \leq g \leq 1. \quad (2)$$

Thus f is normalized and g measures the fraction of reference blackbody intensity associated with absorption coefficients up to k . The inverse cumulative distribution $k(g; \phi, T_r)$ is sampled at the fixed quadrature nodes g_i to produce the stored κ_i . To see how this distribution enters a radiative transfer equation (RTE) solution, consider a stationary, absorbing and emitting, nonscattering medium in local thermodynamic equilibrium. Along a ray with path coordinate s , the spectral RTE is

$$\frac{dI_\eta(s)}{ds} = \kappa_\eta(\phi) [I_{b\eta}(T_g) - I_\eta(s)]. \quad (3)$$

First take a homogeneous segment, so that ϕ and the reference-weighted mapping remain fixed along the ray. Define the intensity per unit cumulative coordinate by

$$I_g(s, g) = \left. \frac{\int_0^\infty I_\eta(s) \delta(k - \kappa_\eta(\phi)) d\eta}{f(k; \phi, T_r)} \right|_{k=k(g; \phi, T_r)}. \quad (4)$$

This is a grouped spectral intensity, not the intensity at one wavenumber. Since $dg = f(k; \phi, T_r) dk$, the total intensity along the ray is $I(s) = \int_0^\infty I_\eta(s) d\eta = \int_0^1 I_g(s, g) dg$. The density notation assumes regular distributions; cumulative bins provide the corresponding interpretation for discrete spectral weights.

Multiplying Eq. (3) by the delta function, integrating over wavenumber, and dividing by the reference distribution gives

$$\frac{dI_g(s, g)}{ds} = k(g; \phi, T_r) [a(g)I_b(T_g) - I_g(s, g)], \quad a(g) = \left. \frac{f(k; \phi, T_g)}{f(k; \phi, T_r)} \right|_{k=k(g; \phi, T_r)}. \quad (5)$$

The a-function converts the reference Planck weighting to the local emission weighting; Section 4.3 develops this correction. At each of the 32 quadrature nodes, the solver therefore advances a gray-gas RTE,

$$\frac{dI_i}{ds} = \kappa_i [a_i I_b(T_g) - I_i], \quad I_i(s) = I_g(s, g_i). \quad (6)$$

$$I(s) \approx \sum_{i=1}^{32} w_i I_i(s). \quad (7)$$

4.3 Reweighting local emission: the a-function

Local thermal emission is weighted at T_g , whereas the table coordinate may be weighted at T_r . For the same local absorption spectrum, define

$$g_r(k) = g(k; \phi, T_r), \quad g_\ell(k) = g(k; \phi, T_g), \quad k_r(g_r) = k(g_r; \phi, T_r).$$

Here k_r is the inverse of $g_r(k)$. The coordinate denoted by g in the preceding RTE equations is this reference coordinate, $g \equiv g_r$, not the local-emission coordinate g_ℓ . Keeping the reference subscript explicit, the normalized Planck-weight correction is

$$a(g_r; \phi, T_g, T_r) = \frac{dg_\ell(k_r(g_r))}{dg_r} = \left. \frac{f(k; \phi, T_g)}{f(k; \phi, T_r)} \right|_{k=k_r(g_r)}, \quad \int_0^1 a(g_r) dg_r = 1, \quad (8)$$